Dike Project Report

Carpooling and Ridesharing Platform
System Analysis, Design, Implementation and Evaluation

PRN232 Enterprise Application Development
Class SEK19B.Net | FPT University Quy Nhon
31 October 2026

Project team

Team member	Student ID
Phan Chi Thông	QE190072
Hồ Lê Quốc Khánh	QE190009
Nguyễn Công Nhất	QE190017
Nguyễn Ái Linh	QE190107
Trần Nguyễn Bảo Long	QE190010

Project advisor: Huynh Vang, Department of Software Engineering.
Email: vangh@fe.edu.vn

Executive overview

Dike connects people who have compatible travel routes and departure times. It organizes the process from publishing a travel request or trip through proposals, booking, communication, trip progress and post-trip feedback. The project combines a Next.js web application, a Java Spring Boot backend and a React Native/Expo mobile application in one repository.

The backend is a modular monolith. MongoDB stores business records, Redis supports sessions and short-lived state, and an outbox worker processes committed events. The web application uses a same-origin proxy for authenticated API requests. The mobile application uses the same API contract with a separate credential transport.

The submitted implementation has automated checks for application logic, architecture and selected browser workflows. On 31 October 2026, the web unit suite passed 23 tests, the backend suite passed 228 tests, and two selected web E2E specifications passed 38 tests across desktop and mobile browser viewports. Workspace typechecking and the OpenAPI contract check also passed. These results do not establish full production readiness or measured capacity for 500 users.

Repository and deployed applications

Repository	https://github.com/hlqkhanh/Dike-v2
Web application	https://dike-v2.vercel.app
Backend API	https://dike-api-demo.onrender.com

Contents

1 Problem and objectives 3

2 Requirements and scope 4

3 System architecture 6

4 Web application design 8

5 Backend and data design 11

6 Mobile application 18

7 Security and privacy 19

8 Deployment 20

9 Verification and results 22

10 Local setup and reproduction 23

11 Limitations and future work 24

12 Project references 25

Reading guide

The report explains requirements and system boundaries, then architecture, web and backend implementation, mobile, security, hosting and verification. Diagram images and worked examples explain important design decisions. Implemented controls and recorded tests are kept separate from unverified readiness claims.

1 Problem and objectives

Informal carpooling coordination can scatter route details, departure times and seat availability across messages. Participants need a shared record of the arrangement and a controlled way to propose, accept, cancel and track a journey. Dike addresses this coordination problem through structured records and explicit state transitions.

The main objectives are to provide a usable web interface; centralize authorization and business rules in the backend; prevent invalid or repeated booking operations; support messaging and change notifications; and maintain a shared, reviewable API contract. A controlled pilot of approximately 500 users is a planning target, not a verified load-test result.

The primary users are visitors, members or passengers, eligible drivers and administrators. Authentication does not automatically grant driver or administrator privileges. Access depends on the user’s current authority, resource ownership and business conditions.

Stakeholders and system boundary

Actor	Primary need	Authority boundary
Visitor	Understand the service and sign in	Public entry only; no private data
Member / passenger	Create travel needs, find offers, communicate and manage participation	Ownership, visibility and participant rules
Eligible driver	Offer approved capacity and manage a journey	Current driver/vehicle eligibility and trip ownership
Administrator	Handle reports and restricted account actions	Explicit privileged authority and audit

The application addresses coordination rather than providing a fleet, operating a taxi service or verifying a bank transfer. A trip offer, a conversation, a proposal and a confirmed booking represent different levels of intent. The interface and data model must preserve those distinctions so that participants do not mistake a discussion for a reserved seat.

The project scope includes local adapters, provider integration boundaries and a free hosted demonstration. Environment configuration changes which capabilities are available. Real SMS delivery, identity/vehicle verification, object storage and physical-device acceptance need evidence beyond the existence of a class or dependency. The report treats implementation, planning targets and executed results as separate categories.

2 Requirements and scope

Functional requirements

ID	Requirement
FR01	Google sign-in, session lookup, logout and session revocation.
FR02	Profile, phone onboarding, privacy, consent and account deletion.
FR03	Friendship requests, friendship management and blocking.
FR04	Recurring schedules, schedule exceptions and travel intents.
FR05	Place selection, route planning and journey estimates.
FR06	Draft, publish, close, inspect and save ride posts.
FR07	Create and manage trips, stops, seats and lifecycle actions.
FR08	Submit, accept, reject or withdraw proposals and track bookings.
FR09	Conversations, messages, notifications and read state.
FR10	Reviews, reports, moderation, account actions and audit.

These requirements describe the application scope and code organization. Provider availability and environment configuration determine which features can be demonstrated on the hosted deployment.

Acceptance structure

Each capability needs a successful path, access checks and explicit exceptions. For example, publishing a driver offer requires a usable owned route, future departure, an eligible driver and a compatible approved vehicle. Creating a readable post is not sufficient if opening the trip or creating its source failed.

For proposal acceptance, the observable result must include coherent seat availability, a booking linked to the proposal, accepted terms and a durable change event. An HTTP response alone does not establish the absence of a competing booking. Repeat, stale-version and final-seat scenarios belong in the acceptance criteria.

Use case	Preconditions and expected result
UC01 Publish an offer	Driver capability and route are valid; source/trip and post are created; opening is explicit; retries preserve completed steps.
UC02 Respond to a post	Open readable post, different owner, valid response input; a versioned decision can open a conversation.
UC03 Confirm a booking	Proposal recipient, live proposal, unchanged terms and available capacity; transaction creates the booking and coordinates related state.
UC04 Execute a journey	Only valid participant actions advance arrival, boarding, start and completion; exceptional closure requires a reason.

Quality requirements

• **Consistency:** changes to bookings, seat availability and related events must be coordinated when accepting a proposal.

• **Security:** the backend must validate identity, authority, ownership and input for each relevant action.

• **Privacy:** profile, post, conversation and location access must follow the applicable visibility rules.

• **Recovery:** clients must be able to refresh authoritative REST data after realtime interruption; background work must tolerate retries.

• **Maintainability:** module boundaries, generated API types and repeatable checks support future changes.

• **Usability:** interfaces must explain loading, empty, failed and expired-session states.

Risk-based interpretation

Consistency requirements target concrete invariants: remaining seats stay between zero and offered seats; one proposal cannot produce two bookings; one travel intent cannot hold multiple active bookings; a cancelled seat cannot be returned repeatedly. Security requirements target current authority and access to the specific resource, not merely whether a user is logged in.

Usability includes partial publication, expired route evidence, provider failure, unavailable session lookup, disconnected realtime and empty match lists. Maintainability requires that shared API types and module boundaries remain reviewable as the application evolves. No numerical performance threshold is reported as achieved without a corresponding measurement.

3 System architecture

Components and communication

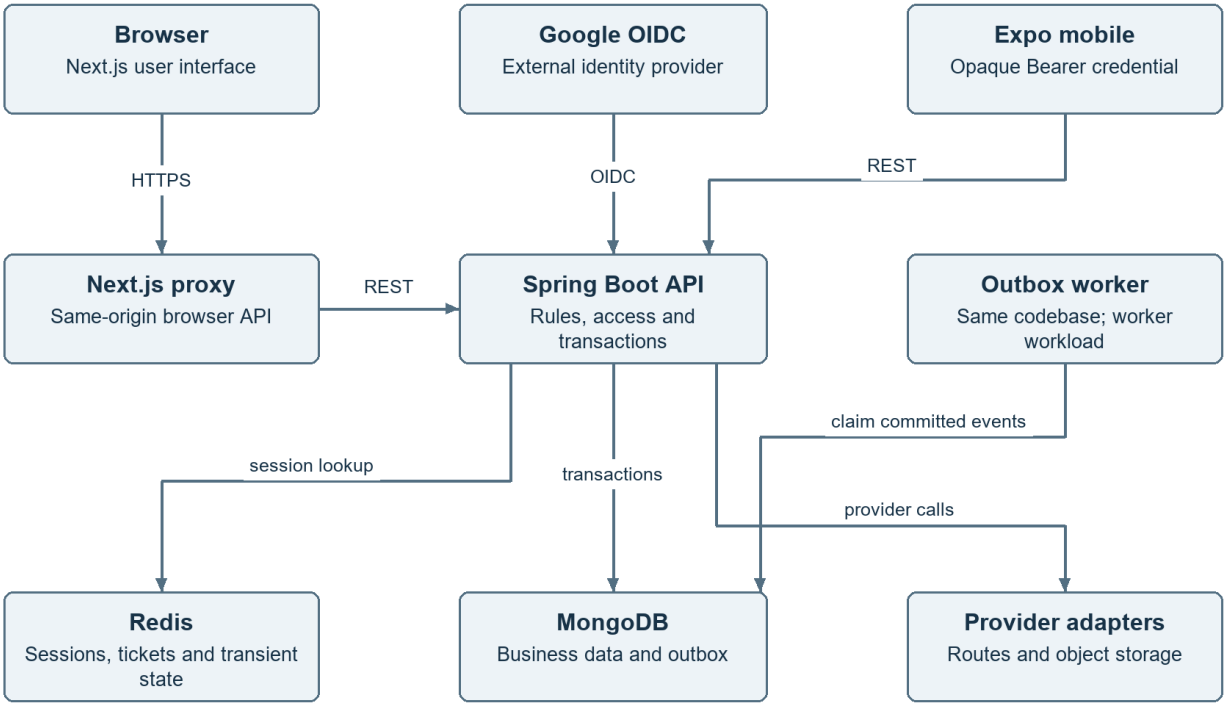


Figure 1 Component architecture and data flow. Provider-backed capabilities depend on the environment.

The API owns authorization, validation and business state changes. MongoDB is the durable store. Redis supports sessions and transient coordination. Object storage has a local S3-compatible implementation and a planned hosted provider configuration; file operations are disabled in the free hosted demo. The API and worker share the same codebase and release cycle.

Why a modular monolith

API and worker use the same artifact and domain model. Keeping business changes within one codebase avoids a separate network release dependency for each capability and makes multi-record transactions easier to reason about. The cost is shared release coordination and less independent scaling than separate services.

MongoDB is the durable business store; Redis is not the booking source of truth. A socket update is a signal that data changed, while REST supplies the authoritative state after reconnect. Provider calls are isolated behind adapters and should not be confused with local database validation. These design choices reduce accidental coupling between presentation, persistence and external network availability.

Technology stack

Layer	Technologies and responsibility
Web	Next.js 16.3.8, React 19.3.0 and TypeScript 6.0.3 for routing and interactive UI.
Client data	TanStack React Query, React Hook Form and Zod for remote state, forms and input validation.
Maps and graphics	MapLibre GL and Three.js for map views and selected visual components.
Backend	Java 21, Spring Boot 4.1.1 and Spring Modulith for a modular monolith.
Persistence	MongoDB for business records; Redis for session and short-lived state.
Realtime	WebSocket/STOMP with client cache refresh and REST synchronization.
Mobile	React Native 0.86.3 and Expo 57 with Expo Router and SecureStore.
Contracts	Springdoc/OpenAPI and generated shared TypeScript types.
Verification	Vitest, Playwright, Axe, JUnit, Mockito, ArchUnit and Testcontainers.
Tooling	pnpm workspaces, Maven Wrapper and Docker Compose.

Versions are taken from the project manifests. They are not claims about the newest available versions.

Repository organization

Path	Responsibility
apps/web	Next.js pages, features and browser tests
apps/backend	Spring API, worker, domain and Java tests
apps/mobile	React Native / Expo application
packages	Shared API, design values, UI and configuration
docs / infra / plan	Contracts, runbooks, deployment and readiness

Development and contract workflow

The pnpm workspace hosts web, mobile and shared packages. Maven builds the backend from the repository root. The reviewed OpenAPI artifact lives under docs/api, and generated TypeScript types support both clients. Contract generation catches drift against that stored artifact; it does not compare every running deployment with the current working tree.

Feature code, domain records, adapters and tests remain close enough to trace a behavior from its screen to its use case and persistence. Repository manifests are the source for reported versions. Architectural tests provide evidence about the tested dependency rules; they do not substitute for tests of a particular trip transition or concurrency invariant.

4 Web application design

Routing and user journeys

The App Router groups pages by URL, while feature-oriented source folders organize authentication, profiles, social relationships, routing, rides and journey logic. The main route groups cover login and onboarding; the feed and post details; schedules; trips; proposals and bookings; messaging; profiles and privacy; and administration.

The unified ride flow provides a common entry point for driver and passenger interactions. Route data and selected pickup information can be carried into the relevant request form. The selected E2E suite checks role-dependent behavior, retained route information and retry behavior after post creation fails. The existence of a route or test does not by itself prove that the entire hosted journey is available.

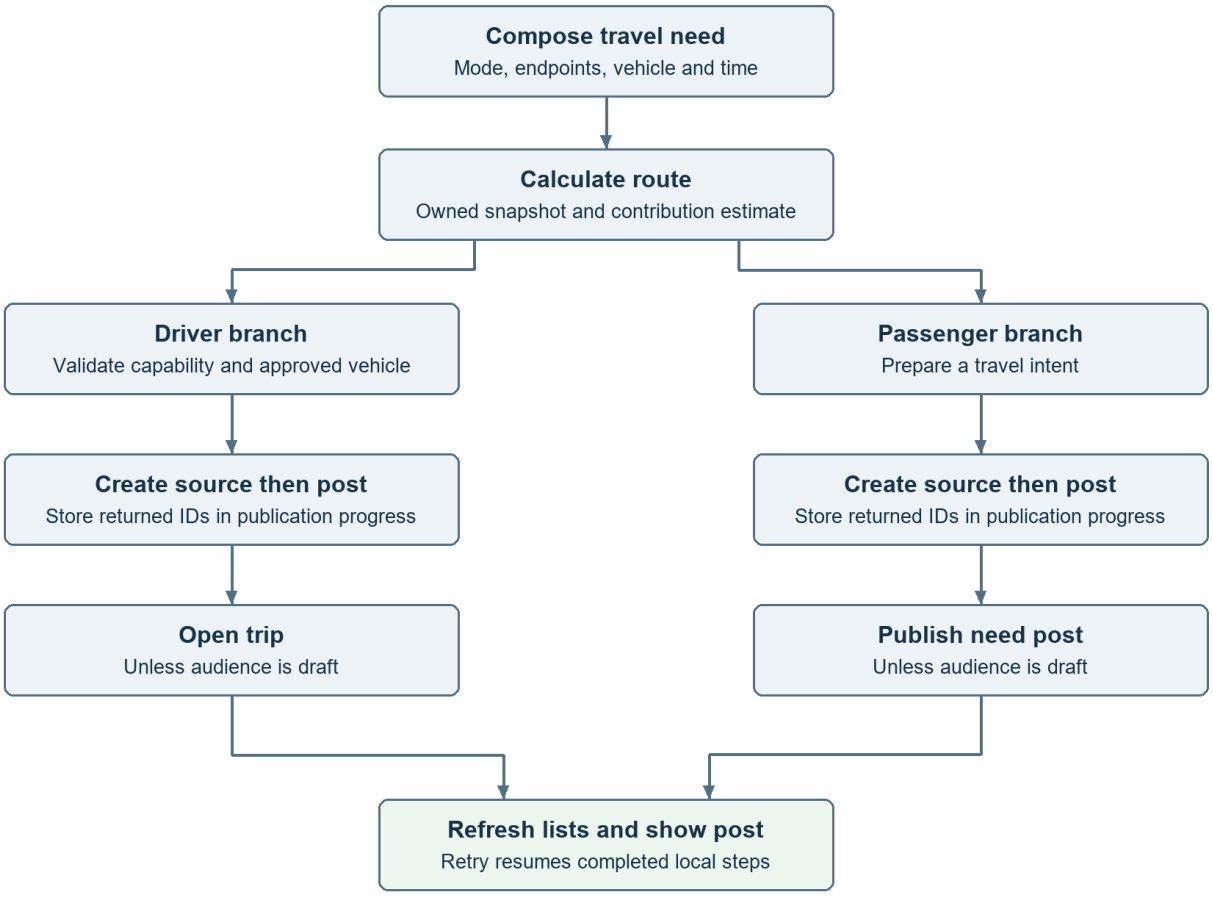


Figure 2 Unified driver/passenger publication. Completed local steps are retained for retry during the mounted flow.

Publication state and partial failure

UnifiedRideFlow keeps a publication reference containing a random command key, departure and returned source/trip/post objects. Driver publication creates the trip first, then the offer post, and finally opens the trip when the audience is not draft. Passenger publication creates a travel source, creates a need post, and publishes it unless draft was selected.

The same publication reference is reused after a later step fails. For example, if the trip exists but post creation failed, retry can reuse the trip rather than deliberately creating another one. An error before any source, trip or post is retained can reset the progress state. Feed and ride query groups are invalidated after success, then navigation opens the resulting post.

This is not one transaction across all browser requests. The progress reference is in memory, not a documented cross-device resume mechanism. A refresh, expired route or changed source version needs an explicit recovery policy. Backend command keys complement this client behavior but have their own actor/operation scope.

Role-specific interaction

The driver path checks capability and compatible vehicle before starting publication. The passenger path can respond to an existing driver offer with pickup context rather than publishing its own draft. Selecting an existing post or carrying an imported route should not silently create a public post. The selected browser tests cover role-dependent forms, route retention and one partial-publication retry scenario.

State and API access

React local state controls form input, tabs and dialogs. React Query manages server data using query keys and refreshes after mutations. The reviewed provider sets a default 30-second stale time and disables automatic query and mutation retries. Authentication failures are handled through shared session-expiry logic. Sensitive operations still require backend validation and idempotency.

Browser requests use the website origin under `/api/...`. Next.js forwards the request to the Spring API using the configured internal API origin. This arrangement keeps the browser-facing authentication path on one origin. Shared TypeScript API definitions help detect contract drift during development.

Client state is not authority

React state describes the current interaction; React Query holds server-derived data under stable query keys. The reviewed provider uses a 30-second default stale time and disables automatic query/mutation retry. Sensitive operations still need server authorization, idempotency and version checks even when the UI disables repeat clicks.

The same-origin proxy forwards browser API calls to the configured internal Spring origin. This keeps the browser-facing session path on the website origin. Generated types reduce shape mismatches, while runtime validation and backend rules remain necessary because a malicious client can bypass the ordinary form.

Failure condition	Expected UI recovery
Session missing or expired	Return to an authentication recovery path and withhold protected content.
Temporary session retrieval error	Show retryable error; do not treat an uncertain session as permission to render private data.
Terms/version conflict	Explain that data changed, reload the authoritative record and require a new decision.
Provider failure	Show explicit unavailability without fabricating route or verification success.
Partially completed publication	Resume known steps where supported, then refresh authoritative lists.

Maps and realtime updates

Map views support place selection, routes and pickup context. The backend must validate route-related conditions that affect a booking; the browser does not decide seat availability or final acceptance conditions. STOMP events notify the UI that relevant data changed. REST refresh remains the synchronization path after a disconnected socket or when a user returns to a page.

Server-owned route and pickup evidence

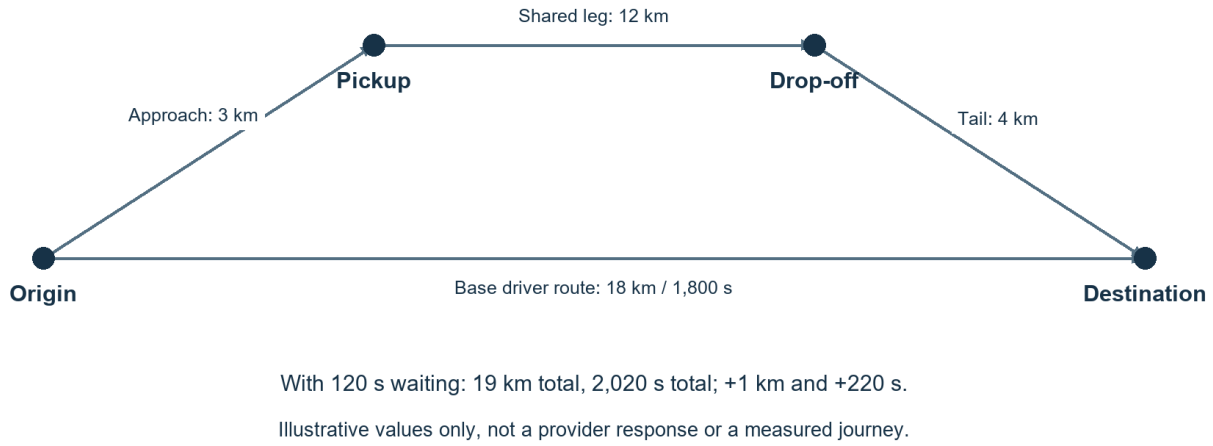


Figure 3 Illustrative routing decomposition. Values explain the calculation and are not provider or performance measurements.

RoutePlanningService resolves owner-bound place references, requests a provider route and stores a route snapshot with provider, mode, endpoints, geometry, distance, duration, timestamps and version. Cache failures are tolerated as optimization failures: they do not turn a valid provider result into invented data or automatically invalidate a successful route response.

PickupCompatibilityService resolves the pickup/drop-off using explicit place references, an optional passenger route or driver endpoints. It constructs an approach leg, the shared passenger leg and a tail to the driver destination, then concatenates geometry and records pickup/drop-off indices. Waiting until passenger readiness increases elapsed time. Extra distance and duration are nonnegative differences against the base driver route.

Worked example

Assume an 18 km / 1,800 s base route. An approach of 3 km / 300 s, shared leg of 12 km / 1,200 s and tail of 4 km / 400 s total 19 km / 1,900 s. With 120 s waiting, total elapsed time is 2,020 s: an extra 1 km and 220 s. At 2,000 VND per shared kilometre, the shared-leg amount is 24,000 VND. These are illustrative arithmetic values, not measured travel times.

RoutePlanningService rounds its contribution suggestion to the nearest 1,000 VND. The pickup-summary estimate uses whole-VND rounding. The two calculations should be reviewed as different layers of price presentation; the agreed booking quote must remain the authority for accepted terms.

Current validation limits

The reviewed compatibility reason is driven by the distance threshold. Returning a latest-pickup window and detour-duration fields does not by itself establish independent enforcement of those time conditions. The pickup record stores a five-minute expiry, but the reviewed reuse method does not explicitly compare expiresAt with the clock. It does validate actor, post, trip and route/version identity.

These are code-review observations, not reproduced production incidents. Expiry cleanup or wider integration must not be assumed to guarantee the missing decision at that call boundary. Targeted tests should exercise expired evidence with stable versions, a late arrival and a duration overrun. The method also rejects adding a pickup when the reviewed trip already has stops, which limits multi-stop matching.

5 Backend and data design

Business modules

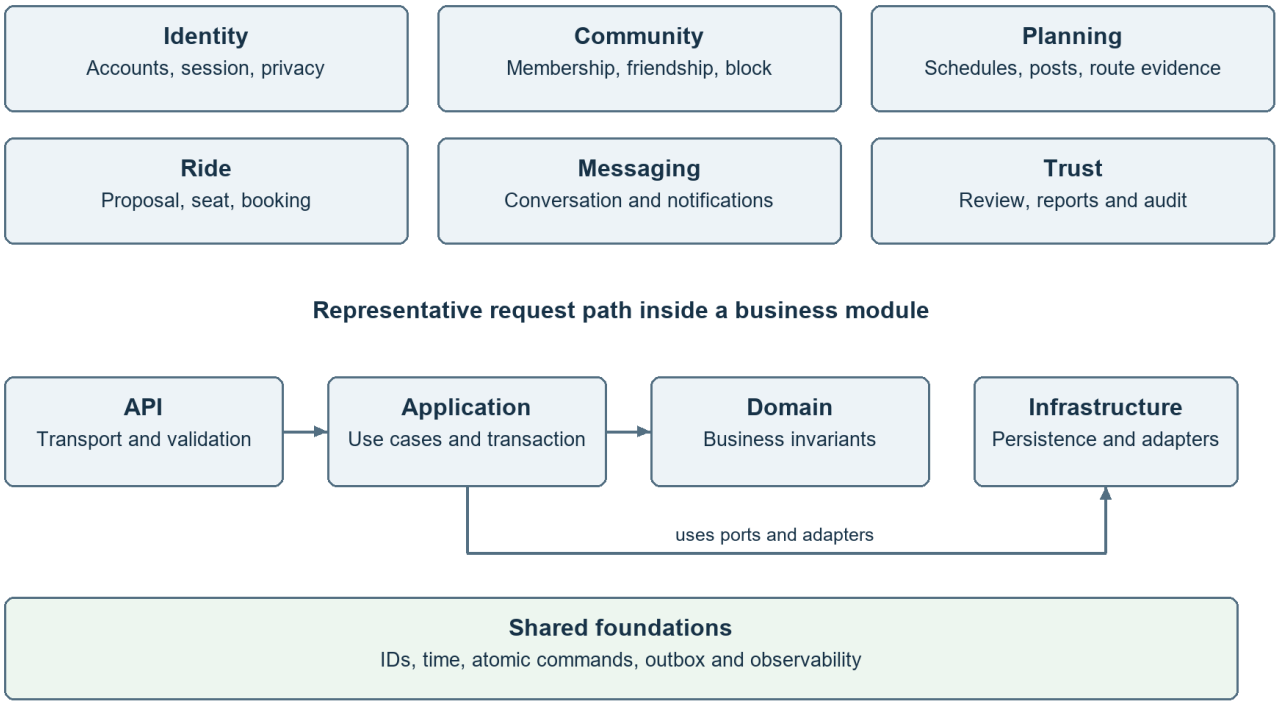


Figure 4 Business module organization and representative internal layers; not separate deployed services.

Module	Responsibility
Identity	Authentication, sessions, OTP, profiles, privacy, consent, files and account deletion.
Community	Community membership, friendships, blocking and social access policy.
Planning	Schedules, travel intents, posts, saved posts, places and route snapshots.
Ride	Trips, stops, proposals, bookings, availability and cost contributions.
Messaging	Conversations, messages, notifications, read state and realtime delivery.
Trust	Reviews, reports, moderation and administrative audit.
Shared	IDs, time, errors, outbox processing and observability foundations.

Each module separates API, application, domain and infrastructure responsibilities. Controllers delegate to application use cases; domain objects enforce state and business rules; infrastructure implements persistence and provider access. Modules communicate through public APIs and published events. Architecture tests check these boundaries.

The internal request path separates transport from use-case orchestration. Domain records own local state/invariant logic, while infrastructure implements persistence and external adapters. Public module APIs and published events connect capabilities. The diagram shows a logical structure, not independently deployed business services.

Shared technical primitives should not become a replacement for business authorization. For example, AtomicCommands manages the transaction and receipt mechanism, but the caller must determine whether the actor may accept a proposal, operate a trip or view a private record.

Core records

Accounts and profiles identify participants and privacy settings. Schedules and travel intents represent recurring or specific travel needs. Posts expose a shareable request or offer with route context. Trips describe the journey, driver, capacity and stops. Proposals represent a request to join; bookings record an accepted arrangement and its progress. Conversations and messages support communication. Reviews, reports and audit records support trust and traceability.

MongoDB stores these business records. Session information is managed separately through Redis. Files are represented by object-storage keys and metadata rather than embedding large binary objects in ordinary profile records.

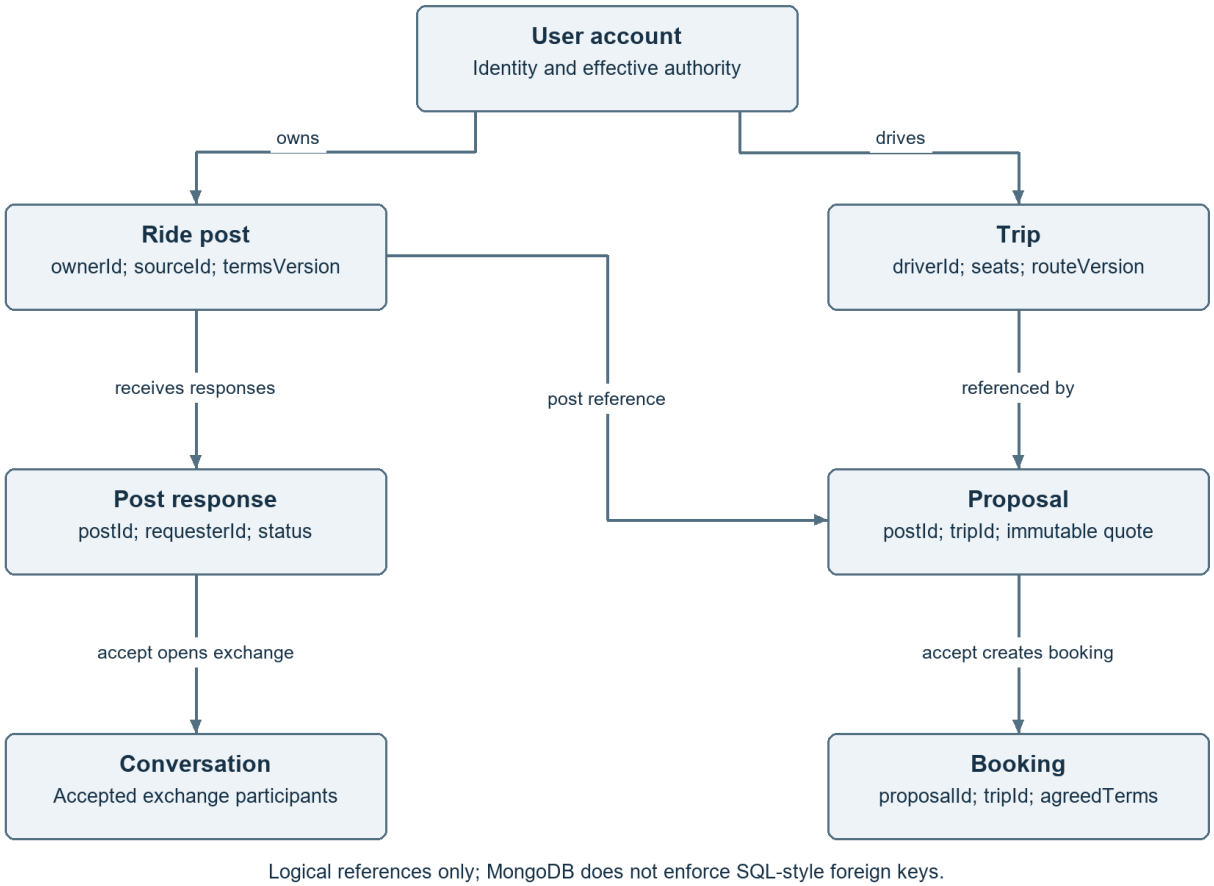


Figure 5 Selected logical references. The discussion track and booking track are distinct; MongoDB does not supply SQL foreign keys.

Record	Important fields / consistency role
Trip	driverId, vehicle/version, route/version, termsVersion, offered/remaining seats, stops and visibility.
Proposal	post/intent/trip, passenger/driver, sender/recipient, direction, immutable quote, expiry and version.
Booking	proposal/post/intent/trip, participants, agreed terms, status, reason, seatReleased and version.
Post response	post/requester/owner, action, status, message, pickup evidence and conversation ID.
Command receipt	Scoped command digest, payload fingerprint, resulting resource ID and expiry timestamp.
Outbox / delivery receipt	Durable event, claim/attempt state and per-consumer completion evidence.

An agreed quote is a historical snapshot rather than a pointer to whatever a provider calculates later. Substituting a current route for accepted terms would change the meaning of the booking. The separate route, terms and record versions help distinguish changes to an arrangement from the original evidence reviewed by the participant.

Indexes and invariants

Constraint	Business effect
Unique booking proposalId	Prevents two booking records from the same proposal.
Active-intent partial uniqueness	Restricts one active booking across CONFIRMED, DRIVER_ARRIVED, BOARDED and IN_PROGRESS.
Pending intent/trip pair	Partial unique index avoids duplicate pending proposal pairs.
Pending passenger request	A partial intent index constrains PASSENGER_REQUEST proposals.
Trip construction	Seats stay within capacity; arrival follows departure; a motorcycle permits one passenger.
User interval index	Supports lookups; a plain index alone does not prove interval non-overlap.

Proposal acceptance and consistency

A typical acceptance operation follows these steps:

1. Authenticate the actor and verify the right to accept the proposal.

2. Load the current proposal, trip and relevant schedule or availability state.

3. Check the current lifecycle state, remaining capacity and business conditions.

4. Apply the accepted booking, seat changes and related outbox event within the transaction boundary.

5. Commit the change and return the authoritative result.

6. Process the outbox and notify clients; clients refresh the affected data.

Version checks and unique indexes help detect conflicting or duplicate changes. Receipt and idempotency mechanisms support repeated requests. Outbox handlers must tolerate delivery retries without repeating the underlying business effect. The associated concurrency integration tests still require execution with Docker before claiming verified behavior under concurrent database operations.

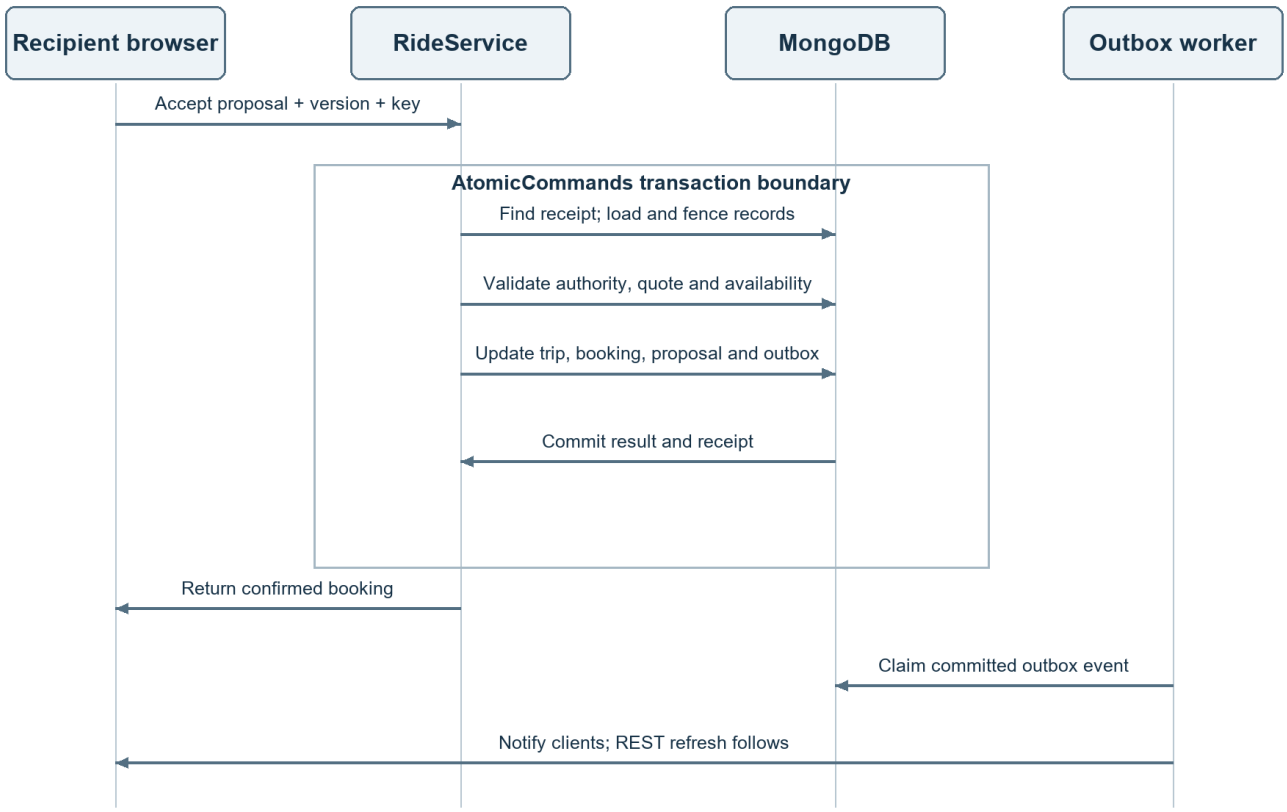


Figure 6 High-level proposal acceptance and committed-event processing. The worker is outside the business transaction.

Detailed acceptance path

RideService.accept scopes the command through AtomicCommands. Inside the transaction, the service loads the proposal, verifies its recipient, returns an existing booking when appropriate, and checks current version and live state. It fences the parties and travel intent, reloads the trip, checks visibility and current driver/vehicle access, and verifies the post terms and intent.

The service rejects an already active travel intent and validates the immutable quote. It reserves driver and passenger intervals; the reviewed path includes a 15-minute end buffer. It validates stops, reserves a seat and updates the trip, inserts the booking, creates the boarding secret, marks the need post booked, updates the offer and accepts the proposal. Competing pending proposals for the same intent are expired.

The booking-confirmed event is part of the durable transaction. A later notification failure must not erase an already committed booking. An uncertain network result should be resolved by fetching the authoritative booking or repeating a supported command, not by assuming the operation failed and creating a second independent arrangement.

Command receipts and concurrent change

AtomicCommands hashes actor, operation and key into a receipt ID and hashes the payload into a fingerprint. A repeated command with matching content returns its prior resource ID. Reusing the scoped key with changed content produces IDEMPOTENCY_CONFLICT. Receipts store a 30-day expiry timestamp.

The transaction helper retries selected duplicate-key, optimistic-lock and Mongo transient/unknown-commit errors up to five attempts, then reports a conflict. Shared guard records coordinate writers that compete on the same invariant. Version checks, guards, indexes and receipts complement one another; they are not a universal exactly-once guarantee.

Evidence boundary

Real concurrent acceptance needs a transaction-capable database and integration execution. The recorded backend unit/architecture suite passed, but Testcontainers was blocked by unavailable Docker. The controls above are implemented mechanisms; this report does not claim an observed zero-overbooking load result.

Lifecycle rules

Trips define draft, open, full, in-progress, completed, cancelled and expired states. Proposals define pending, accepted, rejected, withdrawn and expired states. Bookings include confirmation, driver arrival, boarding, progress, completion and exceptional termination states. These values do not permit arbitrary transitions: the actor, action and current state determine which transition is valid.

Boarding codes are provided only to the appropriate participant and are consumed after successful verification. Cancellation and seat release must avoid returning the same seat more than once. Post-trip cost-contribution records capture participant responses; they are not an integrated payment gateway or a verified bank transaction.

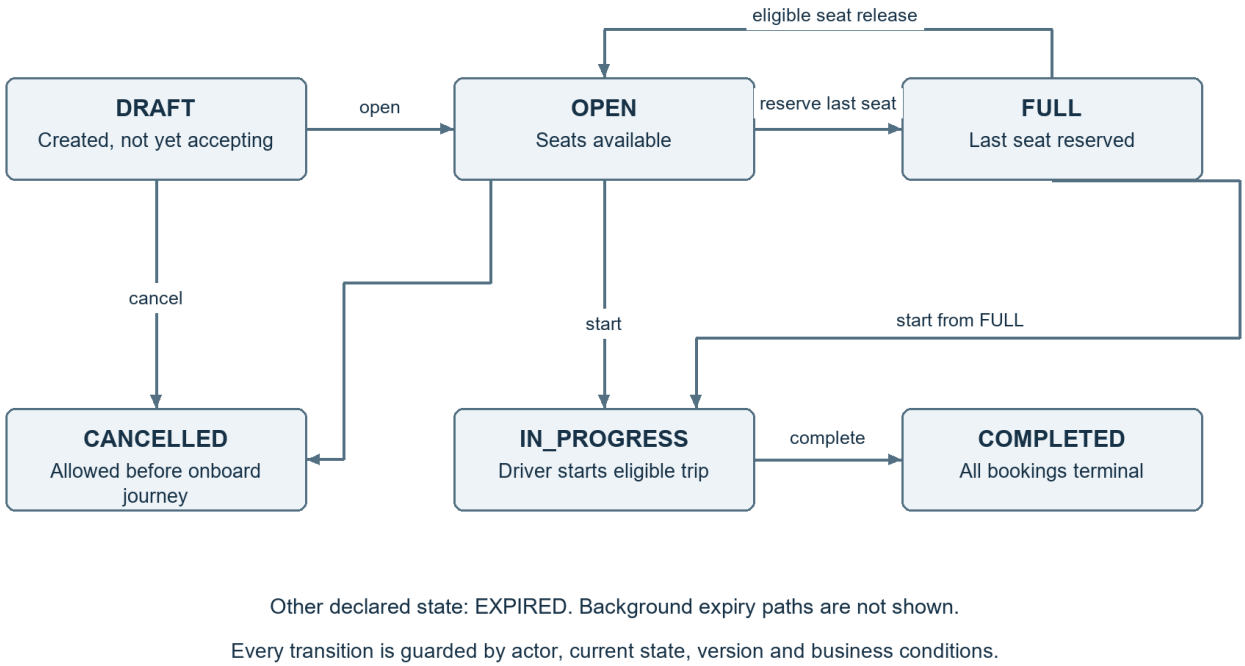


Figure 7 Main reviewed trip transitions. The declared expiry state and some exceptional paths are omitted from this selective graph.

Action	Selected guard or effect
Start trip	Owned current version; OPEN or FULL; not more than 15 minutes before departure.
Arrive / board	Driver actions from the permitted booking states; boarding secret is consumed.
Complete booking	Driver transition from IN_PROGRESS.
Cancel / no-show	Explanation required; no-show cannot occur before ten minutes after agreed pickup.
Return a seat	Only once for eligible predeparture cancellation/no-show while trip is OPEN or FULL.
Complete trip	Trip must be IN_PROGRESS and all bookings terminal.

Booking states and trip states are separate. Starting a trip advances boarded bookings; completion of the trip is blocked while a booking remains active. A seatReleased marker prevents repeatedly restoring capacity.

Cancellation, no-show and ending early have different timing and actor conditions, so an enum list alone is not a state-transition specification.

Post responses are a separate commitment track

Aspect	Post response	Formal proposal / booking
Purpose	Express interest and coordinate an exchange	Accept a validated travel arrangement
Acceptance effect	Versioned response update and participant conversation	Transactional seat reservation and booking
Evidence	Pickup check where required	Immutable quote, current terms, expiry and versions
Success means	Accepted exchange state	Confirmed booking with agreed terms

RidePostResponseService requires an open readable post, rejects self-response and limits the message to 300 characters. Existing active responses can be reused or a pending response updated with a version/status condition. The owner/requester roles determine who may accept, reject or withdraw. Acceptance opens a conversation and stores its ID.

For REQUEST_RIDE acceptance, the service revalidates pickup evidence. Its progress values include DRIVER_STARTED, DRIVER_ARRIVED and COMPLETED, but those are response-track values. They do not by themselves prove that the stronger RideService.accept booking transaction executed. A future bridge or UI should make the commitment level explicit.

Messaging and event recovery

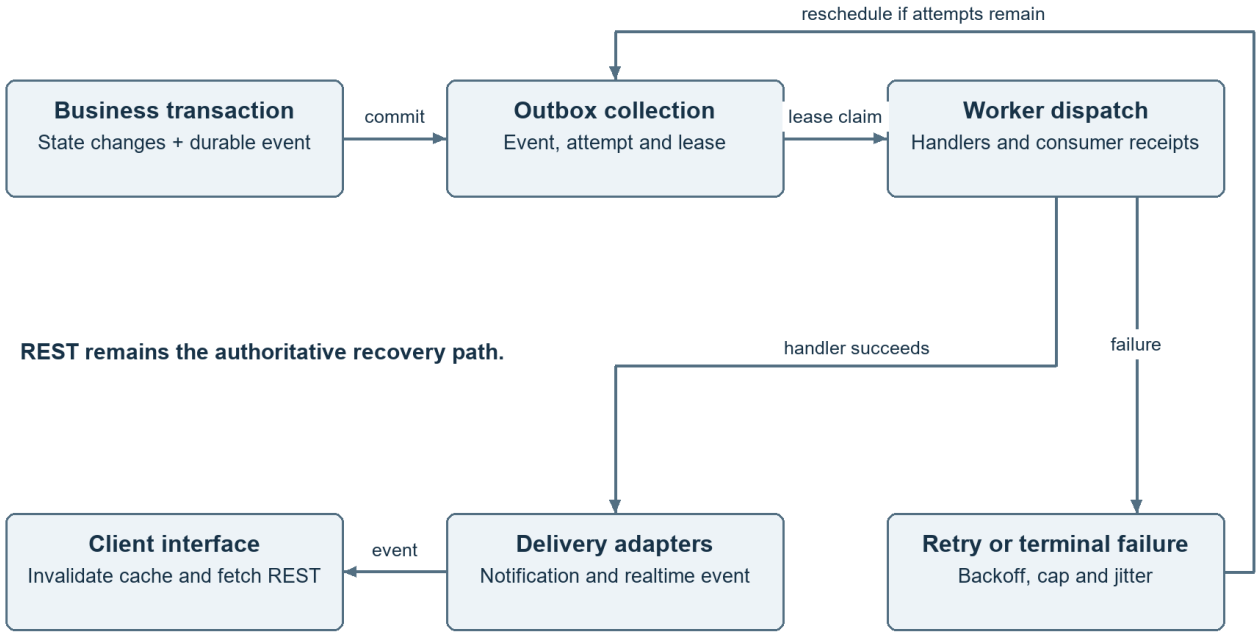


Figure 8 Outbox dispatch, per-consumer receipts and recovery. REST remains authoritative after missed realtime signals.

The worker claims a lease on committed events, invokes registered handlers and skips consumers already marked complete. It records delivery failures and reschedules with capped exponential backoff and jitter, or terminally fails after the configured attempt limit. Claimed/retried/failed counters make the pipeline observable.

This supports recovery between a successful database transaction and delayed notification. Handlers still need idempotency at meaningful boundaries: a receipt does not make an arbitrary external side effect exactly-once. Lease expiry, partial handler success and restart are integration scenarios requiring fault injection.

RealtimeTicketService creates a random short-lived ticket, stores a digest-keyed Redis value with authorization version, and consumes it with get-and-delete. Membership and access checks remain necessary for messages and subscriptions. Socket events may invalidate cached data; REST retrieves the current authorized state.

6 Mobile application

The Expo application shares the API contract and backend with the website. Its screens include the feed, post details, ride composition, inbox and profile. Map-related components present route context, journey stops and pickup requests. Mobile credentials are stored through SecureStore and sent as opaque Bearer credentials; the web continues to use its cookie-based session transport.

Development login is available only in the permitted local/test configuration. It must not become a public hosted login bypass. Mobile Google authentication and real-device acceptance remain work to complete. Typechecking is not an Android or iOS functional test. Secure storage, push notifications and background location require a suitable Development Build and device-based verification; Expo Go is not an acceptance substitute.

Concern	Verification boundary
Secure credential transport	SecureStore and opaque Bearer transport are separate from browser cookie sessions.
Development login	Local/test only; it must not become a hosted public bypass.
Navigation and session state	Restart, expiry, revocation, deep link and background return need acceptance.
Location and maps	Provider errors, denied permissions and device loading behavior require real-device checks.
Push / background behavior	Installed dependencies do not prove permission, delivery or battery behavior.
Static contract checks	Typechecking helps integration but is not physical-device functional acceptance.

A physical-device plan should include denied permissions, no connectivity, app restart, session revocation, background return and slow map loading. Use a suitable Development Build for secure storage, notifications and location integration. Expo Go is not a substitute for every native acceptance scenario.

The recorded mobile-related success is workspace typechecking. The browser project called mobile Chromium is a viewport, not the Expo application on Android or iOS. Complete mobile Google login, battery use, frame rate and cold-start time remain unverified.

7 Security and privacy

The backend validates the account and effective authority on relevant requests. It also checks the actor’s relationship to the target resource. A hidden frontend button or protected page improves navigation but cannot replace server authorization.

Browser state-changing requests are protected through the configured CSRF, Origin and cookie policies. Local authentication and simulated permissions remain environment-restricted. Profile visibility, conversations and location data require access checks that also consider blocking and membership rules.

Private files are accessed through controlled references or temporary URLs after authorization. Credentials, session tokens and personal account passwords are excluded from the submitted documents. Provider integrations, retention behavior and account-deletion handling need further operational acceptance before a real pilot.

Boundary	Control / acceptance consideration
Identity and authority	OIDC identity plus current first-party session and effective access; sign-in is not driver/admin approval.
Resource targeting	Ownership, participant membership, blocking and community visibility must hold for each action.
Browser requests	Configured trusted-origin, CSRF and cookie rules protect mutation paths.
Replay and conflict	Command fingerprint, current versions and unique indexes complement access checks.
Realtime admission	Random ticket, TTL, digest-keyed storage and one-time consumption in the reviewed ticket service.
Sensitive data	Keep tokens, detailed personal fields and unnecessary provider payloads out of logs and exports.

Threat-oriented analysis

A client can modify a hidden field, target another member identifier, replay an old record version or reuse a command key with different input. The backend must reject these cases even if the ordinary screen cannot generate them. Social blocking or community membership can change between evidence creation and acceptance, so stale frontend permissions cannot authorize the operation.

HTTP-only cookies reduce direct JavaScript credential access, not every XSS or request-forgery risk. Similarly, one-time websocket admission is not proof that every subscription is authorized. Passing unit tests establishes their assertions, not immunity to an entire vulnerability class.

Location privacy and deletion

Endpoints, contact details and journey times can identify participants. Public area labels and detailed private information should be separated according to policy, with temporary or controlled access to private files. Retention, object cleanup and backup deletion require operating exercises; configuration files are not proof that a deletion completed across all stores.

8 Deployment

Hosted demonstration

The frontend is deployed on Vercel and the backend on Render Free. The demo uses the `demo,combined` profile so the API and worker share one process. MongoDB Atlas and the configured Redis service support the deployment. The repository contains `infra/render-demo.yaml` and an operational runbook for this topology.

Web	https://dike-v2.vercel.app
API	https://dike-api-demo.onrender.com
Liveness	https://dike-api-demo.onrender.com/actuator/health/liveness
Readiness	https://dike-api-demo.onrender.com/actuator/health/readiness
OpenAPI	https://dike-api-demo.onrender.com/api/openapi.json

At approximately 17:32 on 31 October 2026, Asia/Saigon time, the recorded check returned HTTP 200 for the web, API liveness, API readiness and OpenAPI endpoints. Both health endpoints returned `status=UP`. These checks demonstrate reachability and health at that time, not complete business acceptance.

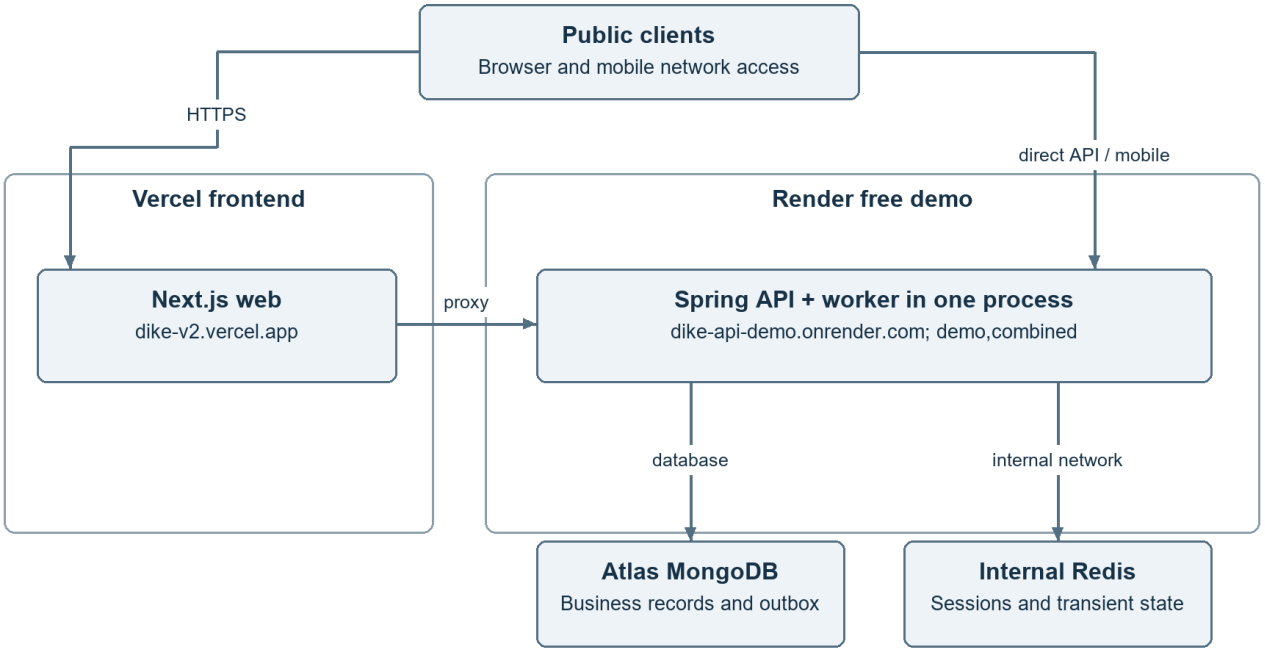


Figure 9 Free hosted demonstration topology. API and worker share one process and its availability.

The demo blueprint selects a Docker web service, readiness health path and demo,combined workload. The combined profile avoids a separate paid worker but couples API and background availability. MongoDB Atlas stores durable records and the configured internal Redis supports sessions/transient state.

A process may become reachable before every business adapter is accepted. Readiness and liveness are useful operating signals, but they do not prove that Google login, a proposal, a message or background delivery completed. Free-service sleep also limits claims about continuous worker operation.

Demo restrictions and version scope

OTP, file operations and routing APIs are intentionally disabled in the free demo. Render Free may sleep when idle, and the worker is paused with the process. Google OAuth is in Testing and may require an approved test user; completion of the Google sign-in journey was not reverified in this preparation run.

The source repository is https://github.com/hlqkhanh/Dike-v2. The recorded branch was `codex/free-demo-deployment`, with HEAD `d46c8916e44f14e0dc844f88817c8ae50abdeac4`. The working tree had uncommitted changes. The submitted GitHub link and deployed service have not been verified to contain every local change. No source copy is included in this submission.

Release and configuration discipline

The frontend origin, upstream API and OAuth callback must agree. A deployment review should record source revision, generated contract, profile, allowed origins and health results together. Secrets belong in provider configuration rather than repository documentation. This report uses recorded checks and does not alter hosted accounts or turn on disabled providers.

The current free demo intentionally disables OTP, storage/file operations and routing APIs. A disabled feature is not a failing production adapter demonstration, and bypassing its guard with local/test authentication would not be valid acceptance. Broader deployment needs the real providers, accepted identity/vehicle policy and operating evidence.

9 Verification and results

Check	Recorded result on 31 October 2026
Web unit tests	PASS: 5 files, 23 tests.
Workspace typechecking	PASS for web, mobile and shared packages.
Stored OpenAPI and generated TypeScript	PASS: contract check exited successfully.
Backend unit and architecture tests	PASS: 228 tests; no failures, errors or skips.
Selected web E2E specifications	PASS: 38 tests across Chromium and mobile Chromium.
Hosted endpoint probes	PASS: HTTP 200; liveness/readiness UP.
Testcontainers integration tests	BLOCKED: Docker Engine unavailable.
Complete E2E, build and overall gate	NOT RUN in the recorded preparation run.
Load test and physical mobile devices	NOT RUN.

The E2E run covered `auth-guard.spec.ts` and `unified-ride-flow.spec.ts`. Their mock API setup verifies browser behavior under controlled responses. It is not a full web–API–database test. The mobile Chromium project is a browser viewport, not an Expo application running on a physical phone. The separate Testing Document gives the test strategy, reproduction commands and planned acceptance scenarios.

Risk coverage and next gates

Risk	Evidence and remaining work
Protected content leakage	Selected auth-guard browser tests passed; complete Google/provider journey still separate.
Partial publication duplication	Selected unified-flow tests passed, including retained route and retry cases; reload recovery needs explicit acceptance.
Final-seat and active-intent races	Implemented transaction/index mechanisms; real Testcontainers concurrency execution still required.
Pickup expiry and time policy	Code-review limits identified; add boundary and stale-evidence scenarios.
Outbox partial failure	Worker unit assertions passed; lease/restart and external-effect recovery need integration.
Performance and mobile hardware	No load, Core Web Vitals, battery/FPS or physical-device acceptance result claimed.

The selected E2E specifications were auth-guard.spec.ts and unified-ride-flow.spec.ts, using two workers across desktop Chromium and mobile Chromium. API mocks isolate browser behavior under controlled responses. A passing result therefore cannot establish Mongo transaction safety or provider integration.

The separate Testing Document records scenarios, reproduction commands and NOT RUN/BLOCKED scopes. Before broader acceptance, run the complete browser suite, a real authenticated API/database journey, Maven verify and the full pnpm check gate on one immutable source snapshot. Do not infer full-gate success from passing components.

10 Local setup and reproduction

The documented prerequisites are Git, Node.js 24.21.0, pnpm 12.9.1, Java 21 and Docker Desktop with Compose. Maven 3.9.10 is supplied by the wrapper. Create local configuration from the example environment file and keep real credentials out of version control.

```
corepack enable
corepack prepare pnpm@12.9.1 --activate
pnpm install --frozen-lockfile
pnpm setup:repo
pnpm dev
```

The development launcher coordinates local infrastructure, API, worker and web. The default web address is `http://localhost:3000`; the direct API uses port 8080. Individual processes can be started with `pnpm infra:up`, `pnpm dev:api`, `pnpm dev:worker` and `pnpm dev:web`.

For verification, run the selected unit and contract commands in the Testing Document. The overall `pnpm check` gate includes formatting, linting, typechecking, tests, build and contract checks. Maven `verify` also needs working Docker infrastructure for integration tests.

Operational rehearsal

Observe readiness, error rates, outbox backlog and claimed/retried/failed counters. A healthy endpoint does not imply a drained queue. Rehearse worker restart, expired lease, provider timeout and a client reconnect while a business operation commits. Each drill should state the expected durable state and how the UI recovers.

A physical mobile device needs an API address reachable over its network rather than its own localhost. Deployment acceptance should also compare frontend/backend revisions and callback/origin settings. Restore, rollback and restart behavior require execution evidence; runbooks describe a procedure rather than certify it has succeeded.

11 Limitations and future work

The remaining work includes real provider acceptance for OTP and file inspection; driver and vehicle verification; full Google authentication acceptance; complete integrated web journeys; and physical Android/iOS checks. The free demo intentionally exposes a narrower feature set than the codebase.

The project has no recorded proof of a 500-user load target, Core Web Vitals scores, battery use, mobile frame rate or cold-start performance in this submission. Backup/restore exercises, rollback rehearsals and operating alerts also need execution-based evidence. Future improvements should be prioritized from these measurements and acceptance results rather than inferred from configuration files alone.

Dike demonstrates how a shared contract, modular backend and explicit trip states can support coordinated carpooling. Its current evidence supports the reported checks and a reachable hosted demonstration. Broader release readiness depends on the unfinished provider, integration and operational acceptance work.

Prioritized follow-up

Priority	Finding	Concrete next step
High	Pickup evidence expiry and time-policy enforcement need explicit review.	Test stable-version expired evidence, late pickup and duration overrun; define the authoritative rejection rule.
High	Response acceptance and formal booking are separate.	Verify the UI explains commitment level and any bridge between both tracks.
High	Real concurrent booking and recovery are not accepted.	Run final-seat, conflicting-intent, unknown-commit and worker-restart scenarios.
Medium	Contribution rounding differs between suggestion and pickup summary.	Define a consistent presentation/agreed-price policy and test its boundaries.
Medium	Hosted source parity is not established.	Pin and accept a revision, then verify web/API/contract alignment.
Medium	Provider, device and operating evidence remains incomplete.	Accept real adapters, physical devices and restoration before expanding access.

The pickup findings are review observations, not confirmed production incidents. Wider cleanup/integration may affect runtime behavior; it should be demonstrated rather than assumed to guarantee a check at the examined call boundary. The roadmap should resolve business-rule ambiguity first, execute integration second, and measure performance/capacity before expanding a pilot.

12 Project references

All references below are files in the submitted GitHub repository:

• `README.md`: prerequisites, setup and development commands.

• `docs/api/openapi.json`: reviewed API contract.

• `docs/architecture/`: architecture, modules and decisions.

• `docs/runbooks/render-vercel-atlas.md`: hosted demo setup and restrictions.

• `docs/runbooks/posts-trips-bookings.md`: post, trip and booking operations.

• `plan/STAGE_17_READINESS.md`: pilot readiness gates.

• `apps/web/package.json`, `apps/mobile/package.json` and `pom.xml`: component manifests.

Implementation evidence used for this expanded report

Topic	Repository source
Web publication	apps/web/app/rides/new/unified-ride-flow.tsx
Route / pickup	planning/application/RoutePlanningService.java and PickupCompatibilityService.java
Post response	planning/application/RidePostResponseService.java
Formal booking	ride/application/RideService.java; shared/application/AtomicCommands.java
Domain / indexes	ride/domain/Trip.java, Proposal.java, Booking.java; ride/infrastructure/RideMigration.java
Session / security	identity/application/SessionApplicationService.java; SecurityConfiguration and TrustedOriginFilter
Event / realtime	shared/outbox/OutboxWorker.java; messaging/RealtimeTicketService.java
Deployment	infra/render-demo.yaml; docs/runbooks/render-vercel-atlas.md
Execution evidence	Recorded unit/typecheck/contract/backend/selected E2E logs and HTTP probes on 31 October 2026

Java source paths above are relative to apps/backend/src/main/java/com/dike unless a complete path is shown. The recorded source branch is codex/free-demo-deployment and HEAD is d46c8916e44f14e0dc844f88817c8ae50abdeac4. The working tree contained uncommitted changes, so HEAD does not identify every local edit reviewed.

Diagram images are logical views reconstructed from those implementation paths. They are not screenshots, measured traces or proof of production behavior. The pickup example is hypothetical. Test counts and hosted results remain the recorded preparation outcomes, not a new claim that the complete system has passed acceptance.